Repository 1 (Coursey deploy on AWS): GitHub - gryffin-uit-alpha/Coursey-devops: Coursey is a web platform designed for real-time learning, where users can access courses and materials instantly and interactively. This repo show how to deploy to production environment. · GitHub

Repository 2 (Coursey deploy on-premise): GitHub - gryffin-uit-alpha/Coursey-on-premise

This article shows you how to migrate the Coursey project from AWS to an On-Premise environment. You should have basic knowledge of Kubernetes, Linux, and DevOps concepts to fully understand this guide.

Transitioning from a managed AWS cloud to a self-hosted on-premise infrastructure requires a complete architectural shift. Before diving into the technical configurations, here is a quick overview of the project's strengths and trade-offs in this lab environment:

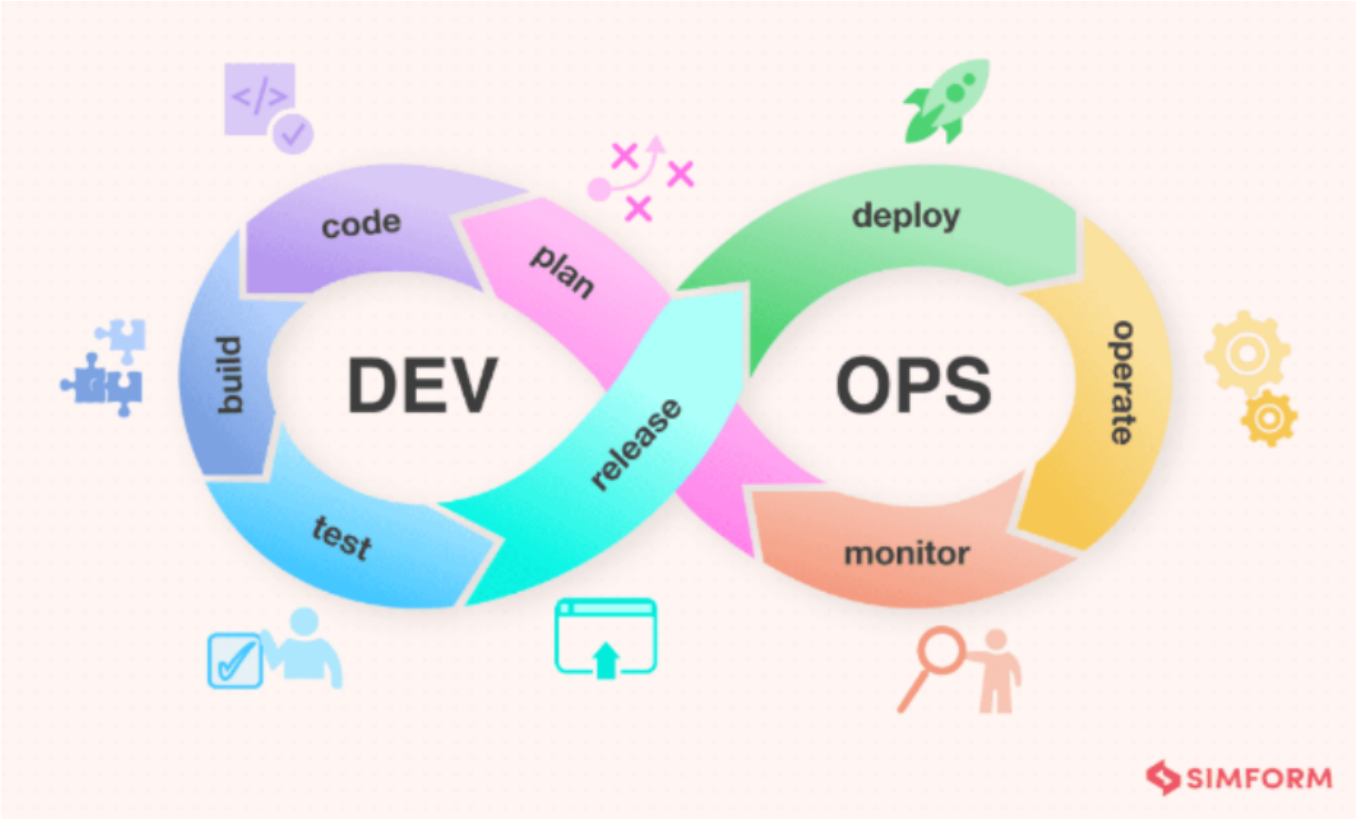
Key Strengths

- **Full Control & Cost Efficiency:** Complete independence from AWS billing. You own and control everything from the physical virtualization network (KVM/Bridge) to the application layer.
- **Modern GitOps CI/CD:** Despite being on-premise, the project utilizes a robust pipeline featuring GitHub Self-hosted Runners for automated builds and ArgoCD for continuous deployment.
- **Advanced Security:** Cloudflare Tunnels are implemented to completely hide the server's public IP, combined with HashiCorp Vault for centralized and encrypted secrets management.

Limitations & Trade-offs

- **No High Availability (HA):** Due to strict hardware resource constraints in the lab environment, the project runs on a Single-node K3s cluster. This introduces a Single Point of Failure (SPOF)—if the VM goes down, the services go down.
- **Limited Infrastructure as Code (IaC):** Unlike AWS, where Terraform can automatically provision the entire network and compute ecosystem, the limited resources here require the initial virtualization (KVM) and networking layers to be configured manually.

AWS to On-Premise K8s (Lab Environment)



Phase 1: Virtual Machine Setup (KVM/QEMU & Virt-manager)

Goal: Set up a virtualization environment on Ubuntu to host Virtual Machines (VMs) acting as servers.

Step 1: Check CPU virtualization support

Before proceeding, ensure your hardware (CPU) supports virtualization and it is enabled in the BIOS.

```
grep -Eoc '(vmx|svm)' /proc/cpuinfo
```

Result: If the command returns a value > 0 (e.g., 12), the machine is virtualization-ready.

Step 2: Install KVM and Virt-manager

Update the system and install the required packages:

```
sudo apt update
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils
virt-manager -y
```

Component	Main Function
qemu-kvm	The primary hypervisor to run VMs.
libvirt-daemon-system	The daemon for managing virtualization processes.
bridge-utils	Tools to configure bridge networking (enabling VMs to share the same subnet as the host).
virt-manager	Graphical User Interface (GUI) for intuitive VM management.

Step 3: Grant management permissions (Without `sudo`)

Add the current user to the virtualization administrative groups:

```
sudo adduser $USER libvirt
sudo adduser $USER kvm
```

Note: You must reboot your machine or log out and log back in for these permissions to take effect.

Step 4: Install Ubuntu Live Server & Configure Static IP

1. **Create VM:** Use Virt-manager to create a new VM with the Ubuntu Server ISO. During installation, ensure you check the "Install OpenSSH Server" option.
2. **Configure Static IP:** After installation, log into the VM and edit the Netplan configuration file:

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

(Pro-Tip applied from our troubleshooting: Removed DNS nameservers to avoid UDP port 53 blockage in the lab environment).

```
network:
  version: 2
  ethernets:
    enp1s0: # Check your VM's interface name using 'ip a'
      dhcp4: no
      addresses:
        - 192.168.122.100/24 # Desired static IP
      routes:
        - to: default
          via: 192.168.122.1 # KVM gateway
```

3. **Apply configuration:** `sudo netplan apply`
4. **Access:** You can now SSH from your host machine into the VM: `ssh user@<your_configured_ip>`.

---

2. **Lightweight Kubernetes (k3s) Installation**

Once the VM is ready, install k3s for container orchestration.

Step 1: Install k3s

Execute the automated installation script provided by Rancher:

```
curl -sfL https://get.k3s.io | sh -
```

Step 2: Configure Access Permissions (Without `sudo`)

To execute `kubectl` commands using a standard user:

```
sudo chmod 644 /etc/rancher/k3s/k3s.yaml
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
```

Tip: Add the export command to your `~/.bashrc` file to persist this setting across terminal sessions:

```
echo 'export KUBECONFIG=/etc/rancher/k3s/k3s.yaml' >> ~/.bashrc
source ~/.bashrc
```

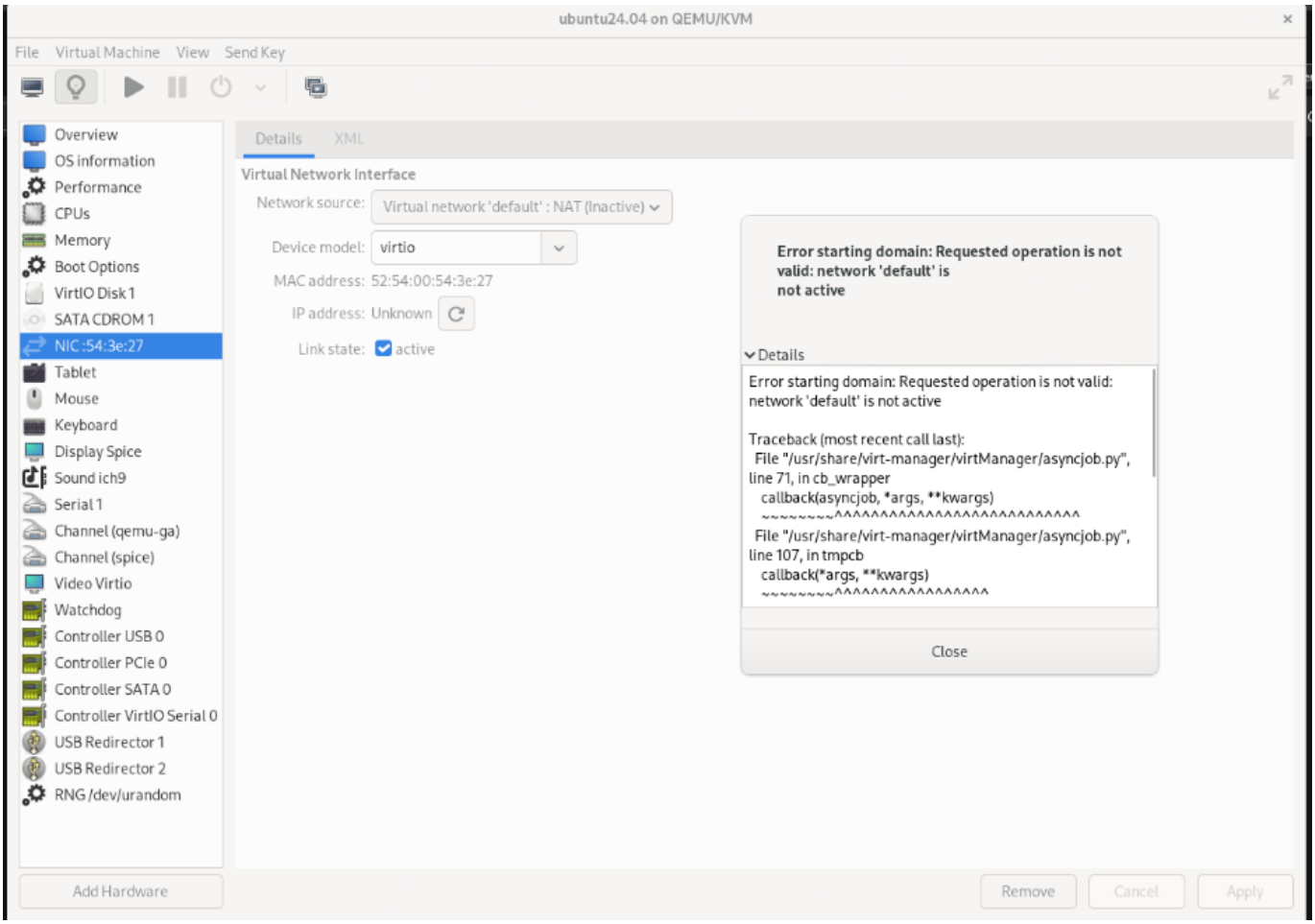
Step 3: Verify System Status

Confirm the k3s cluster is operating correctly:

```
kubectl get nodes
```

Expected Result: The node status should be `Ready`.

Troubleshooting: KVM Network Error



- **Error Details:** `Error starting domain: Requested operation is not valid: network 'default' is not active`.
- **Root Cause:** On certain Debian/Ubuntu distributions, Libvirt's `default` virtual network is not configured to start automatically upon boot (Autostart). Consequently, after a physical host reboot, the KVM system is ready but the virtual network remains inactive.
- **Resolution:**
  1. Check network status:

     ```
     sudo virsh net-list --all
     ```

(The output will show the 'default' network state as 'inactive').

2. Start the network manually:

```
sudo virsh net-start default
```

3. Enable Autostart on boot:

```
sudo virsh net-autostart default
```

Phase 2: Core Services (Networking, Storage, Registry)

1. **Cloudflare Tunnel Installation (Replacing Route 53/ALB)**

Step 1: Install Cloudflare and Login

Download and install the Cloudflare daemon (`cloudflared`):

```
curl -L --output cloudflared.deb
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-
linux-amd64.deb
sudo dpkg -i cloudflared.deb
```

Authenticate the tunnel with your Cloudflare account:

```
cloudflared tunnel login
```

Note: A browser window will prompt you to authorize the domain. Upon successful authorization, a `cert.pem` file will be downloaded to your server.

Step 2: Create a Tunnel for the Project

```
cloudflared tunnel create <tunnel_name>
```

```
gryffin@coursey: ~
https://dash.cloudflare.com/argotunnel?aud=&callback=https%3A%2F%2Flogin.cloudfl
areaccess.org%2FmLxgyFHUdRMHgQl-CCOaKMXgVlszAHRxIhARTV27mVQ%3D

Leave cloudflared running to download the cert automatically.
2026-03-08T01:47:07Z INF You have successfully logged in.
If you wish to copy your credentials to a server, they have been saved to:
/home/gryffin/.cloudflared/cert.pem

gryffin@coursey:~$ ls
cloudflared.deb
gryffin@coursey:~$ pwd
/home/gryffin
gryffin@coursey:~$ ls -a
.               .bash_logout  .cloudflared     .profile
..              .bashrc       cloudflared.deb  .ssh
.bash_history   .cache        .kube            .sudo_as_admin_successful
gryffin@coursey:~$ cloudflared tunnel create coursey-tunnel
Tunnel credentials written to /home/gryffin/.cloudflared/66fef224-20a8-4f33-b130
-de9d2d14f810.json. cloudflared chose this file based on where your origin certi
ficate was found. Keep this file secret. To revoke these credentials, delete the
 tunnel.

Created tunnel coursey-tunnel with id 66fef224-20a8-4f33-b130-de9d2d14f810
gryffin@coursey:~$
```

Make sure to save the generated Tunnel ID for later configuration.

---

2. **Nginx Ingress Controller & Cert-manager Configuration**

Prerequisite: Install Helm if not already available:

```
sudo snap install helm --classic
```

Step 1: Deploy Nginx Ingress Controller

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace
```

Troubleshooting "Cluster Unreachable": If you encounter a connection refused error, set up the `KUBECONFIG` environment variable properly for your standard user:

```
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown -R $USER:$USER ~/.kube
export KUBECONFIG=~/.kube/config
```

Step 2: Install Cert-manager

Used for automating SSL/TLS certificate provisioning.

```
helm repo add jetstack https://charts.jetstack.io
helm repo update
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --set installCRDs=true
```

Step 3: Configure Cloudflare Tunnel (Ingress)

Edit the `cloudflared` configuration file to route traffic to the Nginx Ingress Controller.

```
nano ~/.cloudflared/config.yml
```

```
tunnel: <tunnel_id>
credentials-file: /home/<user>/.cloudflared/<tunnel_id>.json
ingress:
  - hostname: yourdomain.com
    service: http://localhost:80
  - hostname: "*.yourdomain.com"
    service: http://localhost:80
  - service: http_status:404
```

Step 4: Configure ClusterIssuer with Cloudflare DNS-01

1. Generate a Cloudflare API Token (`Zone - DNS - Edit` and `Zone - Zone - Read` permissions).

   Truy cập Cloudflare Dashboard -> My Profile -> API Tokens.

   Chọn Create Token -> Template Edit zone DNS.

   Permissions: Zone - DNS - Edit và Zone - Zone - Read.

   Zone Resources: Include -> Specific zone -> Chọn domain.

2. Create a Kubernetes Secret to store the token:

```
kubectl create secret generic cloudflare-api-token-secret \
  --from-literal=api-token=<CLOUDFLARE_API_TOKEN> \
  -n cert-manager
```

3. Apply the `cluster-issuer.yaml`:

```
nano cluster-issuer.yaml
```

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-cloudflare
spec:
  acme:
    email: <your_email>
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: letsencrypt-cloudflare-account-key
    solvers:
      - dns01:
          cloudflare:
            apiTokenSecretRef:
              name: cloudflare-api-token-secret
              key: api-token
```

Note: If you have two domains and want one domain to host services and the other to host applications, you must register another domain. You can use a shared tunnel; modify the Cloudflare API Token in step 4, instead of "specific zone," select "all zones from an account," then select "account."

Step 5: Setup Cloudflared as a System Service

Move configurations to `/etc/cloudflared/` and install the service to ensure the tunnel runs as a background daemon across reboots.

```
sudo cloudflared service install
sudo systemctl start cloudflared
sudo systemctl enable cloudflared
```

3. **Harbor (Registry) & Local Path Provisioner (Storage)**

Note: Local Path Provisioner is installed by default with K3s.

Step 1: Install Harbor

```
helm repo add harbor https://helm.goharbor.io
helm repo update
```

Create a `harbor-values.yaml` file to configure Ingress, TLS, and Persistence (using `local-path`). Disable Notary to save VM memory.

```yaml
expose:
  type: ingress
  tls:
    enabled: true
    certSource: secret
    secret:
      secretName: "harbor-tls-secret"
  ingress:
    hosts:
      core: harbor.<domain>
    className: nginx
    annotations:
      cert-manager.io/cluster-issuer: "letsencrypt-cloudflare"

externalURL: https://harbor.<domain>

notary:
  enabled: false

persistence:
  persistentVolumeClaim:
    registry:
      storageClass: "local-path"
      size: 10Gi
    chartmuseum:
      storageClass: "local-path"
      size: 2Gi
    jobservice:
      storageClass: "local-path"
      size: 1Gi
    database:
      storageClass: "local-path"
      size: 2Gi
    redis:
      storageClass: "local-path"
      size: 1Gi
    trivy:
      storageClass: "local-path"
      size: 2Gi
```

```
helm install harbor harbor/harbor -n harbor --create-namespace -f harbor-values.yaml [cite: 1444]
```

Troubleshooting Routing Conflicts:

- **Error 404 Not Found:** Traefik (built-in K3s ingress) conflicts with Nginx. **Fix:** Disable Traefik by appending `- traefik` to the `disable:` section in `/etc/rancher/k3s/config.yaml`.

  ```
  sudo nano /etc/rancher/k3s/config.yaml
  ```

  Then paste:

  ```
  disable:
    - traefik
  ```

- **Error 502 / Too Many Redirects:** Cloudflare forces HTTPS (Strict mode), causing an endless redirect loop when hitting Nginx on port 80.

  Fix: Update `/etc/cloudflared/config.yml` to route to `https://localhost:443` and add `noTLSVerify: true` under `originRequest`.

  ```
  sudo nano /etc/cloudflared/config.yml
  ```

  ```
  tunnel: <tunnel_id>

  credentials-file: /etc/cloudflared/<tunnel_id>.json
  ```

ingress:

```
- hostname: <domain>

  service: https://localhost:443

  originRequest:

    noTLSVerify: true

- hostname: "*.<domain>"

  service: https://localhost:443

  originRequest:

    noTLSVerify: true

- service: http_status:404
```

```
Restart Cloudflare

```bash
sudo systemctl restart cloudflared
```
```

---

4. **HashiCorp Vault Installation (Secrets Management)**

```
nano vault-values.yaml
```

```
server:
  dataStorage:
    storageClass: "local-path"
    size: 1Gi

  ingress:
    enabled: true
    ingressClassName: "nginx"
    annotations:
      cert-manager.io/cluster-issuer: "letsencrypt-cloudflare"
    hosts:
      - host: vault.<domain>
ui:
  enabled: true
```

Step 1: Deploy Vault via Helm

```
helm repo add hashicorp https://helm.releases.hashicorp.com
helm install vault hashicorp/vault -n vault --create-namespace -f vault-
values.yaml
```

Step 2: Initialize and Unseal Vault (Shamir's Secret Sharing)

Vault starts in a sealed state to protect master keys.

```
kubectl exec -it vault-0 -n vault -- vault operator init
```

CRITICAL: Save the 5 Unseal Keys and the Initial Root Token generated from the command above.*

Run the unseal command 3 times, providing a different Unseal Key each time to unlock the vault (`Sealed: false`).

```
kubectl exec -it vault-0 -n vault -- vault operator unseal
```

Now can go to vault. to login, use the **initial token** to login

---

5. **SonarQube Installation**

Check server prerequisites: Self-hosted SonarQube is extremely resource-intensive because it utilizes the Elastic Search engine. It requires a large amount of virtual memory, at least 3GB of RAM, and 10GB of storage dedicated solely to SonarQube. Furthermore, you must increase the virtual memory limit and save this configuration to avoid setting it up again after every server restart.

```
sudo sysctl -w vm.max_map_count=262144
echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Meaning: Set the new virtual memory limit to 262144 / Save the configuration. Reloads `/etc/sysctl.conf` immediately.

Step 1: Configure and Install SonarQube

Add the Helm repository:

```
helm repo add sonarqube https://SonarSource.github.io/helm-chart-sonarqube
helm repo update
```

Create the configuration file `sonar-values.yaml`:

```
community: # Use community edition
  enabled: true
ingress:
  enabled: true
  ingressClassName: nginx
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-cloudflare"
  hosts:
    - name: sonar.<domain>
      path: /
  tls:
    - secretName: sonar-tls-secret
      hosts:
        - sonar.<domain>
persistence:
  enabled: true
  storageClass: "local-path"
  size: 5Gi
postgresql:
  persistence:
    enabled: true
    storageClass: "local-path"
    size: 5Gi
```

Step 2: Deploy SonarQube to K3s

```
helm install sonarqube sonarqube/sonarqube -n sonarqube --create-namespace -f
sonar-values.yaml
```

Monitor the progress, as it will take some time.

```
kubectl get pods -n sonarqube -w
```

Once the pod is ready, go to `https://sonar.<domain>` to verify. The default credentials are `admin/admin`. At this point, the dashboard will display the DevOps Platform setup. Choose

"Create a local project" to test immediately, or "Import from GitHub" (takes longer to set up but is required eventually).

Step 3: Test SonarQube with a Local Project

Select "Local project", enter a name, leave default settings, and click Next.

1. SonarQube will ask "How do you want to analyze your repository?". Select **Locally**, then generate a token. Make sure to save this token, as it is only used to scan this specific project.
2. Install `sonar-scanner-cli` on your laptop (where the source code is located, not inside the server. The server only runs the application, it does not hold the source code).

Move to the temporary directory:

```
cd /tmp
```

Download the Sonar Scanner tool:

```
wget https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-scanner-cli-6.1.0.4477-linux-x64.zip
```

(If it times out, use `wget -4 ...` instead) Install `unzip` if not already available, and extract the file:

```
sudo apt-get install unzip -y
unzip sonar-scanner-cli-6.1.0.4477-linux-x64.zip
```

Move the software folder to `/opt/` for system-wide access:

```
sudo mv sonar-scanner-6.1.0.4477-linux-x64 /opt/sonar-scanner
```

Add an environment variable so the system recognizes the `sonar-scanner` command anywhere:

```
echo 'export PATH=$PATH:/opt/sonar-scanner/bin' >> ~/.bashrc
source ~/.bashrc
```

Verify the installation:

```
sonar-scanner -v
```

3. Run the scan to test SonarQube.

   Navigate to the directory containing all your source code. Paste the configuration snippet provided in the SonarQube UI (after generating the token and selecting options) into the command line:

```
sonar-scanner \
  -Dsonar.projectKey=<Project-key> \
  -Dsonar.sources=. \
  -Dsonar.host.url=https://sonar.<domain> \
  -Dsonar.token=<Project-token>
```

Wait for the results on the Sonar Dashboard.

---

2. **Setup SonarQube with GitHub**

Step 1: Setup GitHub App

- Log into GitHub, click your Avatar in the top right corner -> **Settings** -> scroll down to the bottom left and select **Developer settings**.
- Select **GitHub Apps** -> Click **New GitHub App**.
- Fill in the following details:
  1. **GitHub App name:** Must be unique.
  2. **Homepage URL:** `https://sonar.<domain>`
  3. **Callback URL:** `https://sonar.<domain>`
  4. Uncheck the **Active** box in the Webhook section (to avoid complications since we use GitHub Actions for triggering).
- Under **Permissions**, grant the following access:
  - **Checks:** Read & write
  - **Commit statuses:** Read & write

    - **Contents:** Read-only
    - **Pull requests:** Read & write
    - **Metadata:** Read-only
- Scroll to the bottom and click **Create GitHub App**.
- On the next screen, save the **App ID** and **Client ID**. Then click **Generate a new client secret** (save this secret) and scroll down to click **Generate a private key**, which will download a `.pem` file to your machine.
- Click **Save changes**.

Step 2: Setup SonarQube Config

- Go to `https://sonar.<domain>` using the admin account.
- Navigate to **Administration** -> **Configuration** -> **DevOps Platform Integrations**.
- Select the **GitHub** tab -> Click **Create configuration**.
- Fill in the information:
    - **Configuration Name:** `GitHub-Coursey`
    - **GitHub API URL:** `https://api.github.com/`
    - **GitHub App ID, Client ID, Client Secret:** Paste the codes saved in Step 1.
    - **Private Key:** Open the downloaded `.pem` file with a Text Editor, copy its entire contents, and paste it here.
- Click **Save**. Now return to the SonarQube homepage, click **Create Project** -> **GitHub**. It will request to install this App into your GitHub account. Select **Authorize**, then proceed to install the App into GitHub.

Step 3: Install App into GitHub

1. **Find the created application:** Open GitHub, go to **Settings** -> **Developer settings** -> **GitHub Apps**. Click on the app name you just created.
2. **Install the application:** Look at the left menu, click **Install App**. The system will display your GitHub account; click the **Install** button.

3. **Select Repositories to scan:** GitHub will ask whether to allow this app to scan "All repositories" or "Only select repositories". It is recommended to choose **Only select repositories**, search, and select the repository containing your project's code.

4. Scroll down and click **Install**.

5. Return to the "create project" page in SonarQube, choose "from GitHub", select the organization, then import the project you selected in the previous step.

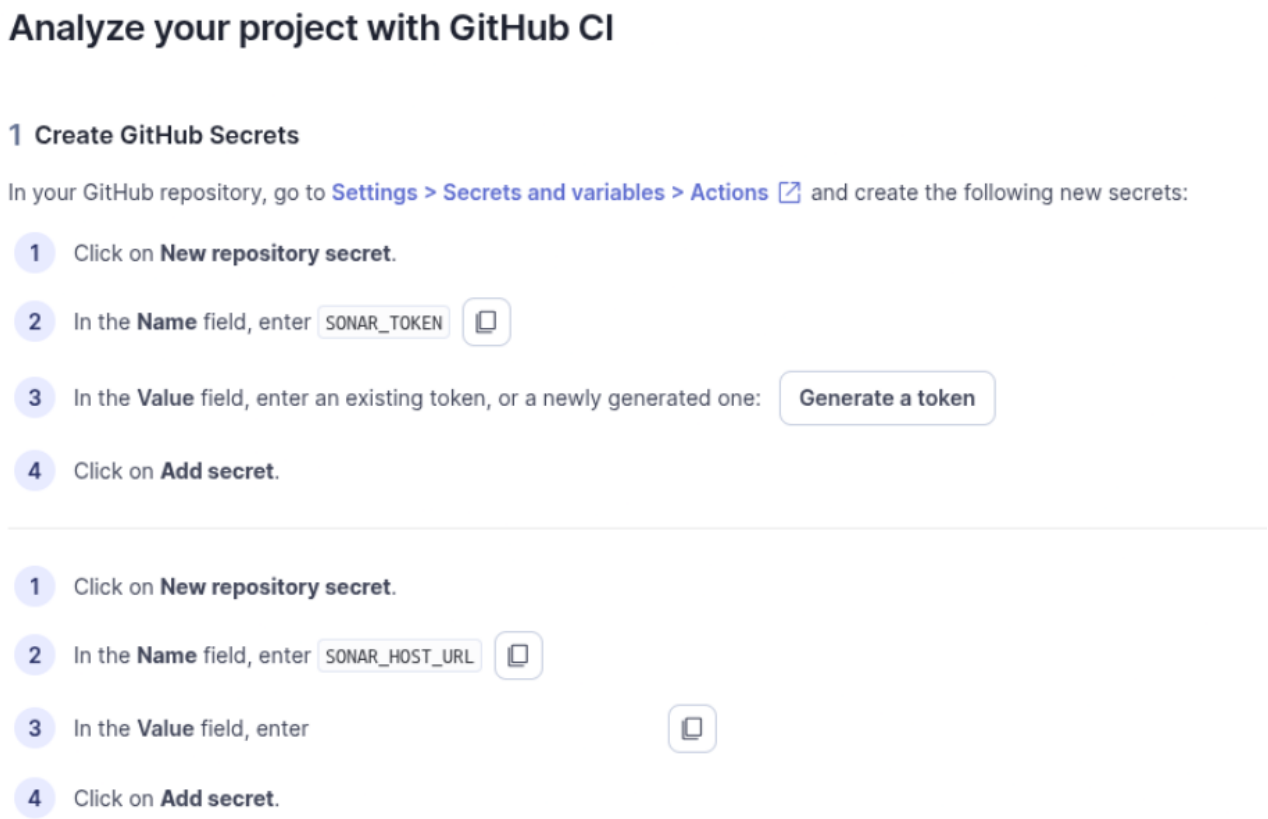
---

3. **Use SonarQube to Scan Project on Push to Main**

Return to Projects, select the project to analyze. SonarQube will ask "How do you want to analyze your repository?", select **GitHub Actions**.

Step 1: Setup repository secrets for GitHub The SonarQube UI will provide instructions, just follow them. Note that the token is only valid for 30 days; it needs to be recreated after this period. In your GitHub repository, go to **Settings** -> **Secrets and variables** -> **Actions** and create the following new secrets:

- `SONAR_TOKEN`: Generate a new token in SonarQube and enter it here.
- `SONAR_HOST_URL`: Enter your SonarQube URL.



Step 2: Setup workflows In your source code, create a file named `sonar-project.properties`:

```
nano sonar-project.properties
```

Paste the content provided by Sonar, which looks like this: `sonar.projectKey=bla...bla...bla.....`

Next, create or update the `.github/workflows/build.yml` file with the content displayed on the SonarQube UI (just click copy). After pasting, push to the `main` branch to see if the workflow runs successfully. Check the SonarQube UI after the project analysis is complete. Done! From now on, any push to main will automatically trigger the CI pipeline.

---

6. **Enhance CI Pipeline & Integrate Harbor to Push Images**

Currently, the CI pipeline includes SonarQube code scanning. We need to update it to build services/applications and push these images to the Harbor registry.

Update: To configure the API URL for the application, create a variable in GitHub Actions:

- **Name:** `REACT_APP_API_URL`
- **Value:** `https://api.<domain>/`

Step 1: Create a Project on Harbor By default, Harbor only has a project named `library`. You should create a new project for your application.

1. Log into Harbor: `https://harbor.<domain>` (default: `admin` / `Harbor12345`).
2. Click the **+ New Project** button.
3. **Project Name:** Name your project.
4. **Access Level:** Leave "Public" unchecked to keep it private (only users with credentials can pull artifacts).
5. **Quota limits:** `-1` means unlimited; you can set a specific limit for your project.
6. Click **OK**.

Step 2: Create Harbor Secrets in GitHub Repository Create two secrets: `HARBOR_USERNAME` and `HARBOR_PASSWORD`. By default, upon Harbor deployment, these are `admin` and `Harbor12345` respectively, unless changed.

Step 3: Write CI pipeline for the project At this stage, when building and pushing images to Harbor, you might encounter an error: `ERROR: received unexpected HTTP status: 502 Bad Gateway`.

- **Cause:** The Cloudflare Free Tier only allows a maximum HTTP request size of 100MB from the client to the server. If exceeded, Cloudflare's edge server drops the connection, leading to a 499 (Client Closed) error in ingress-nginx logs and a `client disconnected during blob PATCH` error in harbor-registry logs.
- **Resolution:** Optimize the image to reduce its size, or upgrade Cloudflare to a paid plan.

You have now covered the Code & Plan, Test, Build, and Release stages of the DevOps Lifecycle.

Phase 3: Migration (Application & Data)

At this step, modify the Helm chart configurations previously used for AWS deployment to suit the on-premise deployment.

1. Store Project Secrets in Vault

1. Log into the Vault UI using the root token.
2. On the left menu, select **Secrets Engines** -> **Enable new engine**.
3. Select **KV** -> **Next**.
4. In the **Path** field, explicitly type: `secret` (Vault defaults to Version 2). Click **Enable Engine**.
5. The interface will move into `secret/`. Click **Create secret**.
6. In the **Path for this secret** field, enter: `coursey/database` (or any path like `asd/abc`, but ensure you update the deployment accordingly).
7. Under **Secret Data**, add the following Key-Value pairs one by one (click Add for new rows):
    - `MYSQL_HOST`: Must match the MySQL service name defined in the `mysql-stateful`.
    - `MYSQL_USER`: Database username.
    - `MYSQL_PASSWORD`: Password.
    - `MYSQL_ROOT_PASSWORD`: MySQL root password.

- `MYSQL_DATABASE`: Database name, must match the database initialized by the `initdb-configmap` (default is `docker-php`). Update both if changed.
- `MAIL_USERNAME`: Email used for the password reset feature.
- `PASS_APP`: Gmail App Password (search for instructions on generating app passwords for Gmail).

8. **Enable K8s Authentication**
    - On the left menu, select **Access** -> **Auth Methods** -> **Enable new method**.
    - Select **Kubernetes** -> **Next**.
    - Keep the default path as `kubernetes`. **Enable Method**.
    - The configuration screen will appear.
    - **Kubernetes host:** Enter `https://kubernetes.default.svc:443` (This is the standard internal address for Vault to call the K3s API to verify Pod identities).
    - Leave other fields (like Kubernetes CA Certificate, Token Reviewer JWT) blank, as Vault deployed via Helm on K8s automatically fetches its own internal certificates.
    - **Save**.

3. Create Vault Policy

On the left menu, select **Policies** -> **ACL Policies** -> **Create ACL policy**.

Name: Give the policy a name.

Policy (HCL): Paste the policy (`data` in the path is required for KV v2):

```
path "secret/data/coursey/database" {
  capabilities = ["read"]
}
```

Create Policy.

4. Create Vault Role

- On the left menu, select **Access** -> **Auth Methods** -> `kubernetes/`.
- Switch to the **Roles** tab -> **Create role**.

- Fill in the following details:
  - **Name:** Role name.
  - **Bound service account names:** Service account name (This is the Service Account you will declare in the YAML file of the service requiring Vault variables).
  - **Bound service account namespaces:** The Kubernetes namespace where the application is deployed.
- Scroll down to the **Tokens** section:
  - Expand the Tokens section, locate **Generated Token's Policies**.
  - Type and select the Policy created in step 3.
- Click **Save**. The Vault password repository setup is now complete.

5. Declare Service Accounts for Pods In the YAML templates, declare a `serviceaccount.yaml` file:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: coursey-backend-sa
  namespace: <your_deployment_namespace>
```

6. Add Annotations to inject secrets from Vault into Deployments. (Refer to the deployment files).

Phase 4: CI/CD Automation (GitOps)

1. Install ArgoCD

Step 1: Create namespace and install ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts -f
https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
```

Check status:

```
kubectl get pods -n argocd
```

At this point, the `dex-server` might show an `ImagePullBackOff` error. *About ArgoCD Dex Server:* Dex is an OPTIONAL auxiliary component. Its sole purpose is to provide Single Sign-On (SSO) allowing logins via Google, GitHub, or Microsoft accounts. You can opt not to deploy this pod:

```
kubectl scale deployment argocd-dex-server --replicas=0 -n argocd
```

Step 2: Configure ArgoCD Ingress

Create `argocd-ingress.yaml`:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: argocd-server-ingress
  namespace: argocd
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-cloudflare"
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
      - argocd.<domain>
      secretName: argocd-tls-secret
  rules:
    - host: argocd.<domain>
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: argocd-server
                port:
                  name: https
```

Apply this configuration to K3s:

```
kubectl apply -f argocd-ingress.yaml
```

Step 3: Log into ArgoCD

By default, ArgoCD generates a random password. Use this command to retrieve it:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="
{.data.password}" | base64 -d; echo
```

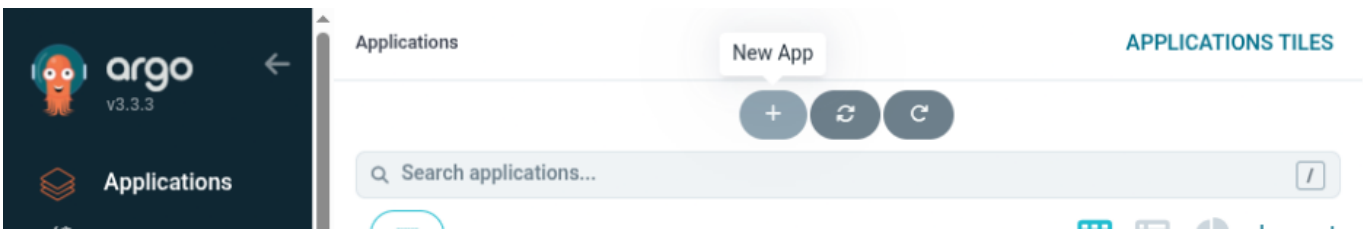
Copy the password, navigate to `https://argocd.<domain>`, and log in (default username: `admin`). You are now in the ArgoCD interface.

2. Connect ArgoCD with Repository

- On the left menu of ArgoCD, select **Settings** -> **Repositories**.
- Click **+ CONNECT REPO USING HTTPS**.
- **Type:** `git`
- **Project:** `default`
- **Repository URL:** Paste your GitHub repo link.
- Click **CONNECT**. It should report Successful. *Note: If the repository is private, you must configure additional permissions for ArgoCD.*

3. Create Application

In the ArgoCD UI, click **+ New App**.



- **GENERAL Section:**
  1. **Application Name:** Set any name.
  2. **Project Name:** `default`
  3. **SYNC POLICY:** Choose `Manual` for the first few tests; switch to `Automatic` once it runs smoothly.
- **SOURCE Section:**

1. **Repository URL:** Your GitHub repository link.
2. **Revision:** `HEAD`
3. **Path:** ArgoCD will automatically scan the directory containing the YAML files.

- **DESTINATION Section:**
  - **Cluster URL:** `https://kubernetes.default.svc` (The internal K3s cluster where ArgoCD resides).
  - **Namespace:** Your application deployment namespace. Click **Create**. To synchronize and deploy to the server, click **SYNC** -> **SYNCHRONIZE**.

    +2

Bug: Pod in ImagePullBackOff Status

- **Cause:** Harbor is a private registry, requiring authorization to pull images.
- **Fix:** Create a secret to log into the registry, then configure deployments to pull images from Harbor.

```
kubectl create secret docker-registry harbor-pull-secret \
  --docker-server=harbor.<domain> \
  --docker-username=admin \
  --docker-password=Harbor12345 \
  --namespace=<deployment_namespace>
```

Update the deployments: Place `imagePullSecrets` inside the `spec` and above `containers`.

```
spec:
  imagePullSecrets:
    - name: harbor-pull-secret
  containers:
    - name: nginx
      image: "{{ .Values.nginx.image.repository }}:{{ .Values.nginx.image.tag
}}"
```

```
    app: coursey-nginx
spec:
  imagePullSecrets:
  - name: harbor-pull-secret
  containers:
  - name: nginx
    image: "{{ .Values.nginx.image.repository }}:{{ .Values.nginx
```

2. GitHub Self-hosted Runner

Why self-hosted? Although GitHub Actions provides runners, a self-hosted runner is necessary for large projects or specific requirements. It allows the use of more powerful hardware (CPU, RAM, GPU), removes CI minute limitations, grants access to internal infrastructure (databases, private APIs, Kubernetes cluster), and helps control the build environment to accelerate pipelines via caching.

Note before installation: Secure your repository if making it public. Go to **Repo -> Settings -> Actions -> General**. Scroll to "Approval running fork pull request workflows from contributors" and check "Require approval for all external contributors".

1. Install Self-hosted Runner

Navigate to your project repository.

- Click the **Settings** tab.
- On the left menu, find **Actions** -> select **Runners**.
- Click the green **New self-hosted runner** button.
- For Runner image, select Linux and x64. Follow the Bash commands provided by GitHub in order.

```
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64-2.332.0.tar.gz -L
https://github.com/actions/runner/releases/download/v2.332.0/actions-runner-
linux-x64-2.332.0.tar.gz
tar xzf ./actions-runner-linux-x64-2.332.0.tar.gz
./config.sh
```

During `./config.sh`, it will ask:

1. *Enter the name of the runner group:* Press Enter or provide a name.

2. *Enter the name of runner:* Name of the runner.
3. *Enter any additional labels:* Press Enter to skip.
4. *Enter name of work folder:* Press Enter (default is `_work`).

If you run `./run.sh`, the bot will die when the server is turned off and must be restarted manually. Instead, set it up as a service:

```
sudo ./svc.sh install
sudo ./svc.sh start
```

Run CI Pipeline with GitHub Self-hosted Runner In your GitHub workflows, change `runs-on: ubuntu-latest` to `runs-on: self-hosted` for any job you want the self-hosted bot to execute.

Note: Docker and Git must be installed on the server prior to this. Git is usually pre-installed; install Docker following official documentation. Trigger the CI pipeline.

Bug: ERROR: permission denied while trying to connect to the docker API...

- **Cause:** The Docker Daemon communicates via the socket `/var/run/docker.sock`. By default, only the root account or users in the `docker` group can access it. The GitHub Runner runs under a standard user, leading to permission errors during `docker build`.
- **Fix:** Add the user to the docker group and restart the bot.

```
sudo usermod -aG docker $USER
newgrp docker
sudo ./svc.sh stop
sudo ./svc.sh start
```

Phase 5: Monitoring (Prometheus and Grafana)

1. Configure Prometheus and Grafana

Create `prom-grafana-values.yaml`:

```
nano prom-grafana-values.yaml
```

```yaml
grafana:
  ingress:
    enabled: true
    ingressClassName: nginx
    annotations:
      cert-manager.io/cluster-issuer: "letsencrypt-cloudflare"
    hosts:
      - grafana.<domain>
    path: /
    tls:
      - secretName: grafana-tls-secret
        hosts:
          - grafana.<domain>
prometheus:
  prometheusSpec:
    retention: 2d
    resources:
      requests:
        cpu: 200m
        memory: 512Mi
      limits:
        cpu: 500m
        memory: 1Gi
```

Then install:

```
helm install prometheus prometheus-community/kube-prometheus-stack -n
monitoring --create-namespace -f prom-grafana-values.yaml
```

2. Login and Monitor the System Retrieve the password:

```
kubectl get secret --namespace monitoring prometheus-grafana -o jsonpath="
{.data.admin-password}" | base64 --decode ; echo
```

Open `https://grafana.<domain>`. The default username is `admin`. In the Grafana UI, open the respective dashboards to view and monitor the system.

6. Conclusion & Future Enhancements

1. Conclusion

The migration of the "Coursey" project from AWS to an On-Premise Kubernetes environment demonstrates a highly successful shift toward a self-managed, cost-effective infrastructure. This

project successfully established a robust virtualization foundation using KVM/QEMU and a ******lightweight K3s cluster. Critical core services were effectively integrated, including Cloudflare Tunnel for secure external access without port forwarding, Nginx Ingress, and Cert-manager for automated SSL provisioning.

Furthermore, the infrastructure was strengthened with a private Harbor registry for artifact management and HashiCorp Vault for centralized, secure secrets management. The development lifecycle was significantly modernized by implementing a GitOps-driven CI/CD pipeline utilizing ArgoCD and GitHub Self-hosted Runners, alongside strict code quality gates powered by SonarQube. Finally, the deployment of the Prometheus and Grafana stack ensures comprehensive observability and monitoring of the entire ecosystem.

2. Future Enhancements

While the current infrastructure is highly functional for a lab environment, there are several avenues for future scaling and optimization:

- High Availability (HA) Architecture: Transition the K3s environment from a single-node setup to a highly available multi-node cluster to eliminate single points of failure.
- Image & Payload Optimization: Further optimize Dockerfile builds to reduce image sizes, mitigating the 100MB payload limit imposed by the Cloudflare free tier during Harbor pushes.
- Advanced Alerting System: Expand the Prometheus and Grafana monitoring stack by configuring Alertmanager to send real-time incident notifications (e.g., via Slack, Telegram, or Email) when system resources reach critical thresholds.
- Automated Disaster Recovery: Implement automated backup strategies (such as Velero) for the K3s cluster state, Longhorn persistent volumes, and the on-premise relational database to ensure data integrity